# ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

**Please do not change the style of this document.**

**Question numbers 1–10 follow the same order as the notebook** (top to bottom).

---

**Student Information**

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name:**
- **Chinese Name:** 蔡承祐
- **Email: RA7141236@gs.ncku.edu.tw**
- **Student ID: RA7141236**
- **University: NCKU(National Cheng Kung University)**

---

## Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

**How to answer each question**

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

**Sources, honesty, and AI use**

**Acceptable vs. Unacceptable AI Usage:** Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

## Questions

**Question 1: Data Filtering & Domain Relevance**

**Notebook:** Section 3.2 — Question 1 (take home)

**Task:** Filter the dataframe **X** for the category **sci.med** and check its length.

**Questions:**

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

**Answer:** 1. 594

2: We should categorize data based on domain relevance, otherwise we would get frequent item sets like "would like", "think about", "also have"…etc. And for efficiency, we reduce noise, which is not domain related, to make our category more precise.

**AI used for this answer:** No

**If yes, how:**

**External references (links):**

---

**Question 2: Handling Missing Data**

**Notebook:** Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

**Task:** In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

**Question:** If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

**Answer:** Just dropping 40% of the dataset with dropna() is bad because we lose almost half of our text and vocabulary, which weakens downstream models. Also, missing labels are usually not completely random, so dropping them introduces bias into the dataset.

An alternative is semi-supervised learning (like pseudo-labeling), where we train on the 60% labeled data to predict labels for the missing ones. Or we can use topic modeling like LDA to group texts without needing labels.

**AI used for this answer:** Yes

**If yes, how:** Brainstormed alternative approaches for handling unlabeled data and checked English grammar.

**External references (links):** Scikit-learn Semi-Supervised Learning: https://scikit-learn.org/stable/modules/semi_supervised.html

Pandas Handling Missing Data Guide: https://pandas.pydata.org/pandas-docs/stable/user_guide/missing_data.html

---

**Question 3: Data Sampling Risks**

**Notebook:** Section 5.1 — Question 3 (take home)

**Task:** In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

**Questions:**

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

**Answer:** 1. No, the proportions do not match the original perfectly due to natural random sampling variation.

2. The main risk is that rare minority categories might get picked very few times or even get completely missed in the sample, so the model won't have enough examples to learn them.

**AI used for this answer:** Yes

**If yes, how:** Looked up sampling risks on imbalanced data and refined conversational wording.

**External references (links):** Scikit-learn Stratified Sampling: https://scikit-learn.org/stable/modules/cross_validation.html#stratified-k-fold

---

**Question 4: Text Vectorization and Matrix Sparsity**

**Notebook:** Section 5.3 — Question 4 (take home)

**Task:** In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

**Questions:**

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

**Answer:**

1. The total number of features increases (from 35,482 to 35,788, adding 306 stop words).

2. Adding stop words back decreases the overall matrix sparsity (from 99.68% to 99.55%). Common words like "the" and "is" appear in almost every document, filling rows with non-zero counts instead of zeros, which makes the matrix less sparse (more dense).

**AI used for this answer:** Yes

**If yes, how:** Verified the sparsity calculation numbers and checked wording.

**External references (links):** Scikit-learn CountVectorizer Reference: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

---

**Question 5: Feature Selection — Variance Thresholding**

**Notebook:** Section 5.3.1 — Question 5 (take home)

**Task:** In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

**Questions:**

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

**Answer:**

| Threshold | Number of terms kept (keep == True) |
|---|---|
| 0.0 | 35482 |
| 0.01 | 5089 |
| 0.05 | 1491 |

1. The vocabulary size drops quickly as the threshold increases: from 35,482 at 0.0, down to 5,089 at 0.01, and only 1,491 at 0.05.
2. Words that only appear once or twice across the whole dataset have near-zero variance, because their count is 0 in almost every document.
3. It is safe because words that almost never appear don't give the model any useful signal to separate categories anyway. Dropping them removes useless noise and speeds up training without hurting accuracy.

**AI used for this answer:** Yes
**If yes, how:** Checked the variance filter results from the notebook and helped phrase the explanations.
**External references (links):**

- Scikit-learn VarianceThreshold Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.VarianceThreshold.html

---

**Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)**

**Notebook:** Section 5.3.2 — Question 6 (take home)

**Task:** Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

**Questions:**

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

**Answer:** Looking at the results, the top Pearson words are mostly generic words like "thanks", "writes", and "looking", while the top Spearman words are domain-specific graphics words like "graphics" (0.52), "image", "files", and "3d".

Mathematically, Pearson measures a linear relationship between raw word counts, which is easily affected by extreme outliers. Spearman ranks the values first and measures whether they move in the same monotonic direction, regardless of linearity. This happens when a word appears consistently in a category but with low counts (like 1 or 2 times per post). Its rank matches the category very well (high Spearman), but because the counts don't scale up linearly, Pearson stays lower.

**AI used for this answer:** Yes
**If yes, how:** Discussed the mathematical difference between Pearson and Spearman rank correlation.
**External references (links):**

- SciPy Spearman Correlation Documentation: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

**Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics**

**Notebook:** Section 5.4.2 — Question 7 (take home)

**Task:** In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

**Questions:**

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

**Answer (1) — Runtime and pattern count when lowering minSup:** Lowering minSup from 9 down to 3 makes runtime much slower and finds a lot more patterns, because lowering the support bar allows many more low-frequency item combinations to qualify.

**Answer (2) — Top-K k vs MaxFPGrowth minSup:** minSup sets a frequency threshold, so you don't know in advance how many patterns will be returned. Parameter k in Top-K sets a fixed limit on the number of returned patterns, giving you exactly the top k most frequent ones.

**Answer (3) — Maximal vs standard frequent patterns:** Standard mining outputs all sub-combinations of a frequent itemset. Maximal frequent patterns only keep the largest itemsets that have no frequent supersets, cutting out all the redundant smaller subsets.

**AI used for this answer:** Yes
**If yes, how:** Refined the comparison between minSup, Top-K, and maximal frequent patterns.
**External references (links):**

- PAMI Library Documentation: https://udaynr.github.io/PAMI/
- Han et al., Mining Frequent Patterns without Candidate Generation (FP-Growth): https://dl.acm.org/doi/10.1145/335191.335372

---

**Question 8: Dimensionality Reduction — Motivation**

**Notebook:** Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

**Task:** After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

**Questions:**

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

**Answer:**

**1.** It is impossible because X_counts has over 35,000 features, and humans can only visualize 2D or 3D plots. We cannot draw a 35,000-axis graph on a screen.

2. We must apply Dimensionality Reduction algorithms first (like PCA, t-SNE, or UMAP) to project the data down to 2D or 3D.

**AI used for this answer:** Yes

**If yes, how:** Brainstormed why high-dimensional text matrices cannot be visualized directly.

**External references (links):**

- Scikit-learn Dimensionality Reduction: https://scikit-learn.org/stable/modules/decomposition.html

---

**Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)**

**Notebook:** Section 5.5 — Question 9 (take home) (after 2D/3D plots)

**Task:** In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

**Questions:**

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

**Answer:**

1. t-SNE and UMAP show clear, separated cluster islands, while PCA shows a heavy overlap around the center.
2. PCA is a linear method that only finds straight axes of maximum variance, so it mashes non-linear text groups together. t-SNE and UMAP are non-linear and focus on keeping local neighbors together, which separates different groups better.
3. In t-SNE, the distance between separate clusters is mostly arbitrary and depends on hyperparameters (like perplexity) and random seed, so physical distance on the plot does not mean true semantic distance.

**AI used for this answer:** Yes

**If yes, how:** Reviewed the difference between linear PCA and non-linear manifold projection.

**External references (links):**

- Van der Maaten & Hinton, Visualizing Data using t-SNE (2008): https://www.jmlr.org/papers/v9/vandermaaten08a.html
- Wattenberg et al., "How to Use t-SNE Effectively", Distill (2016): https://distill.pub/2016/misread-tsne/

---

**Question 10: One-Hot Binarization**

**Notebook:** Section 5.6 — Question 10 (take home)

**Task:** In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

**Questions:**

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

**Answer:**

1. Exactly 4 binary columns are created (one for each category).
2. If we pass raw integers (0, 1, 2, 3), the algorithm wrongly assumes false mathematical relationships:
   - **False Ranking:** It assumes an ordering like 0 < 1 < 2 < 3, implying that Category 3 is somehow "greater" than 0.
   - **False Distance:** It assumes Category 0 is closer to 1 (difference of 1) than to 3 (difference of 3).
3. One-hot binarization gives each category its own separate binary column, making all categories orthogonal and equidistant with no false numerical ranking.

**AI used for this answer:** Yes

**If yes, how:** Refined the explanation of false ordinal assumptions in clear English.
**External references (links):**

- Scikit-learn LabelBinarizer: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html
- Feature Engineering for Machine Learning (Categorical Data): https://towardsdatascience.com/understanding-feature-engineering-part-2-categorical-data-f54324193e63

---